DEVIXO SOLUTIONS

Web Development Internship

Full Stack Student Management System

Task 03 - Project Report

Student Name	Hassan Asad
Internship Domain	Web Development
Organization	Devixo Solutions
Task Number	03
Technology Stack	React, Node.js, Express, MongoDB
Report Date	25 October 2026

Week 3: Advanced Concepts and Real-World Project Development

Project Overview

Objective

Develop a full-stack Student Management System with account authentication, role-based access, and permanent database storage. The application lets administrators maintain student records while users can browse and search the directory.

Technologies Used

Technology	Purpose
React and Vite	Build and run the frontend interface.
CSS	Responsive layouts, Tangerine & Navy themes, and hover effects.
Node.js and Express	Run the backend API and process application requests.
MongoDB Atlas	Store students, user accounts, and login sessions.
bcryptjs	Hash account passwords and verify login attempts.
express-session and connect-mongo	Maintain login sessions using a cookie and MongoDB storage.
dotenv	Load private backend configuration from an environment file.

Website / Application Structure

client/src/App.jsx - Dashboard, student forms, search, and pagination.
client/src/AuthGate.jsx - Registration, login, logout, and session checks.
client/src/UI.jsx - Shared branding, icons, and saved theme toggle.
client/src/api.js - Shared API request and response handling.
client/src/index.css - Styling and responsive breakpoints.
server/index.js - Student routes and database startup.
server/auth.js - Authentication, sessions, and role protection.
server/studentValidation.js - Student field and business-rule validation.

Application Flow

React interface → Express API → MongoDB Atlas. The backend validates each write request, checks access permissions, saves the record, and returns a response that updates the interface.

Implementation and Features

Implemented Features

Module	Implemented behavior
Authentication	Register, login, logout, hashed passwords, and persistent sessions.
Student management	Add, edit, delete, search, and view details. Student IDs are unique and stored in uppercase.
Dashboard	Total students, registrations in the last seven days, distinct courses, and the five newest records.
Responsive interface	Desktop table, mobile record layout, form validation, loading feedback, and empty/error states.
Pagination (bonus)	Client-side pagination with five records per page and numbered page controls.
Roles (bonus)	Admins can add, edit, and delete. Users can view and search. Express enforces these permissions.
Theme enhancement	Light and dark modes with saved preference, matching graduation branding, and minimal hover effects.

Database and Student Fields

The student_management database uses students, users, and sessions collections. Student records contain Student ID, full name, email, course, semester/year, status, and registration timestamp. MongoDB also creates an internal record ID. A partial unique index protects assigned Student IDs while allowing older records without that field.

Validation and Access Rules

Required fields and email format are checked in the interface and backend. Semester choices are 1-8; year choices are 1-4. Status choices are Graduated, Semester Freeze, Active, and On Leave. Under this project's four-year model, Graduated is allowed only for Semester 8 or Year 4. Selecting the final level does not automatically graduate a student.

New accounts start as Users. An administrator sets the initial Admin role in Atlas. Database credentials and the session secret remain in a private .env file and are excluded from Git. The project runs locally and has no deployed demo URL.

Authentication Screens

Figure 1. Login screen

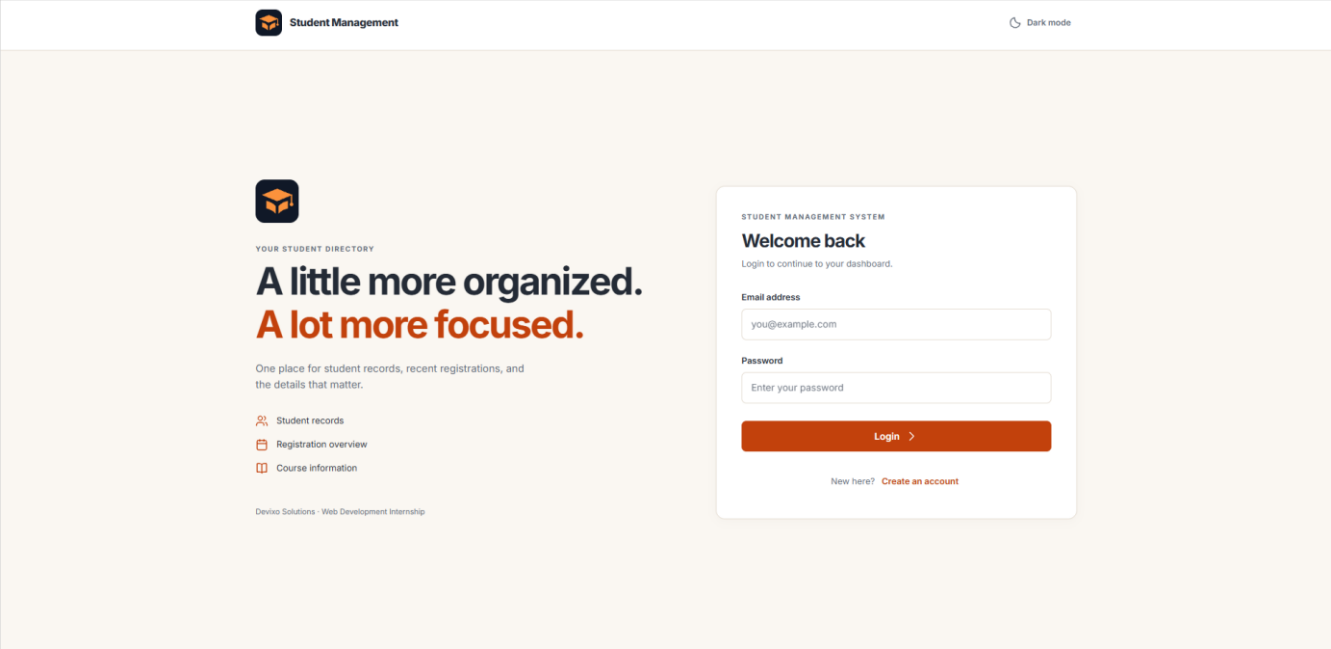
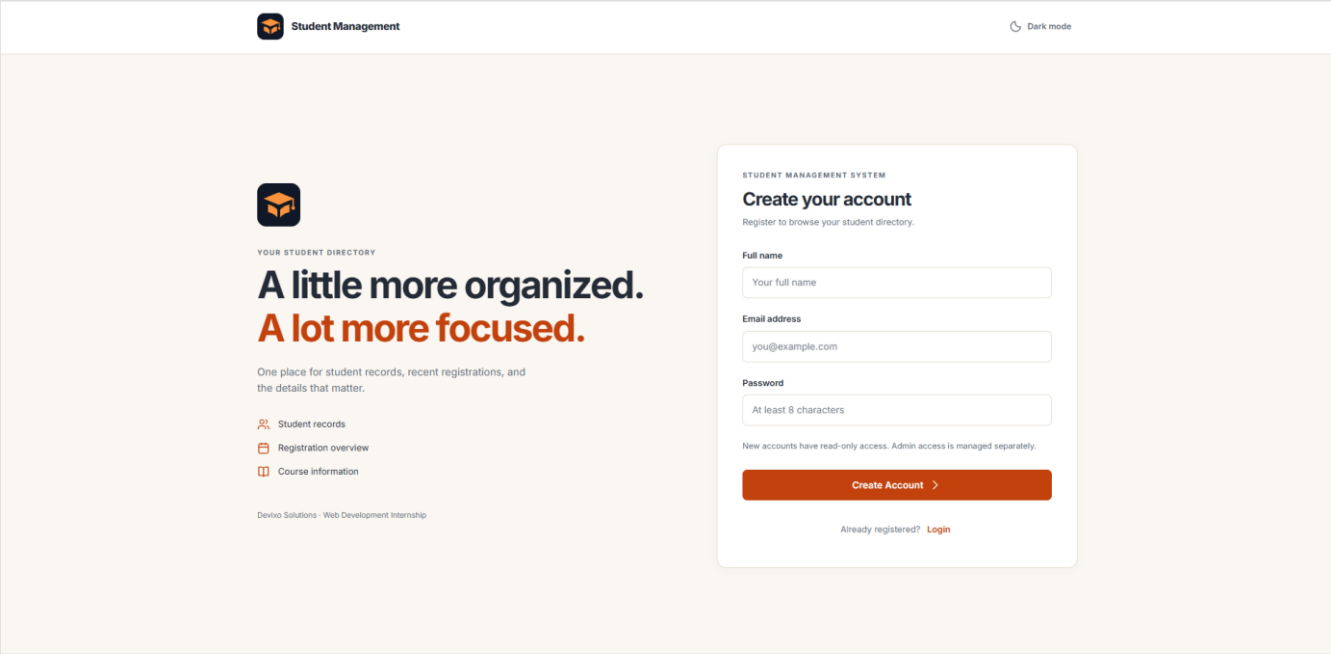


Figure 2. Registration screen



Registration creates a User account. Login checks the account credentials and starts a server-side session.

Dashboard and Role-Based Access

Figure 3. User dashboard with read-only access

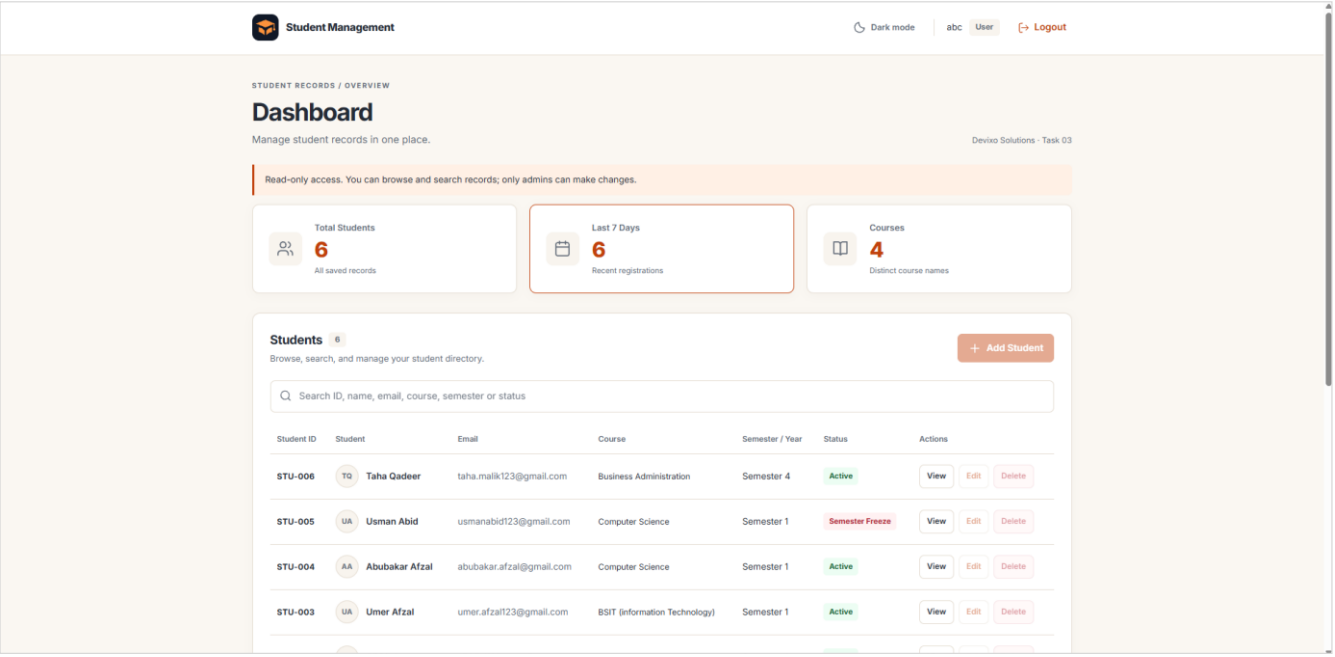
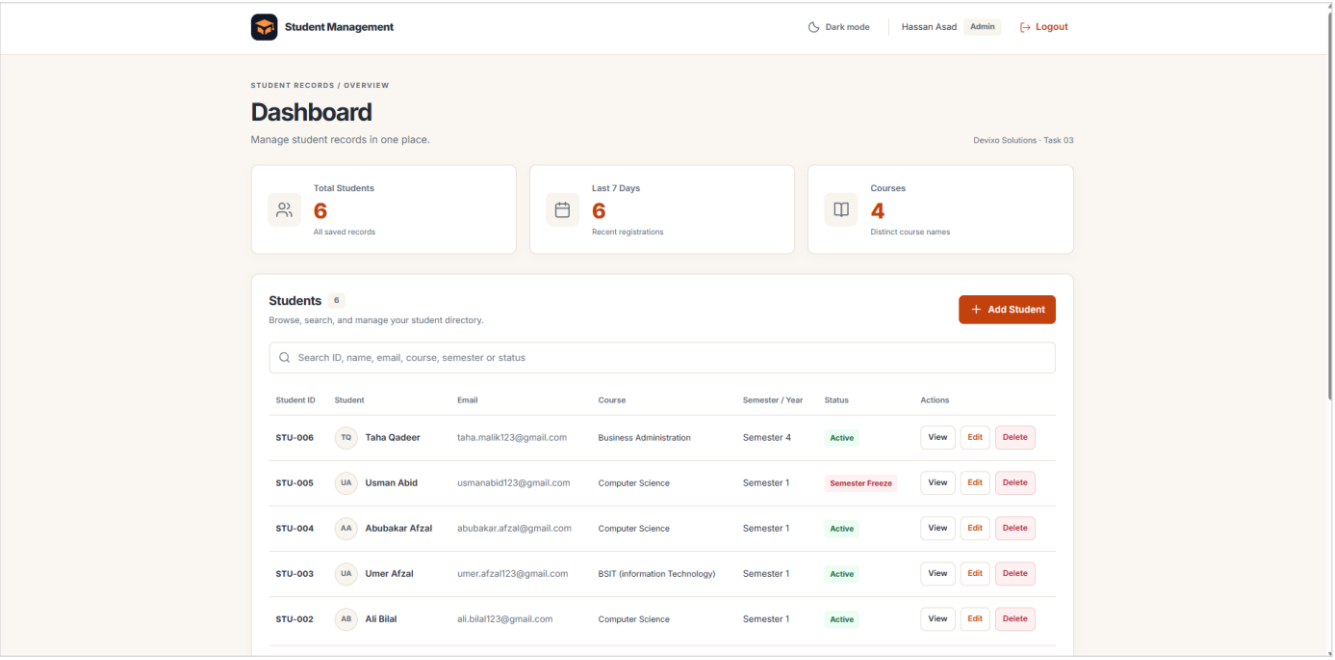


Figure 4. Admin dashboard and statistics cards



Users can browse records. Admins can also add, edit, and delete students. The dashboard summarizes student totals, recent registrations, and courses.

Student Directory and Pagination

Figure 5a. Student directory, search, and page controls

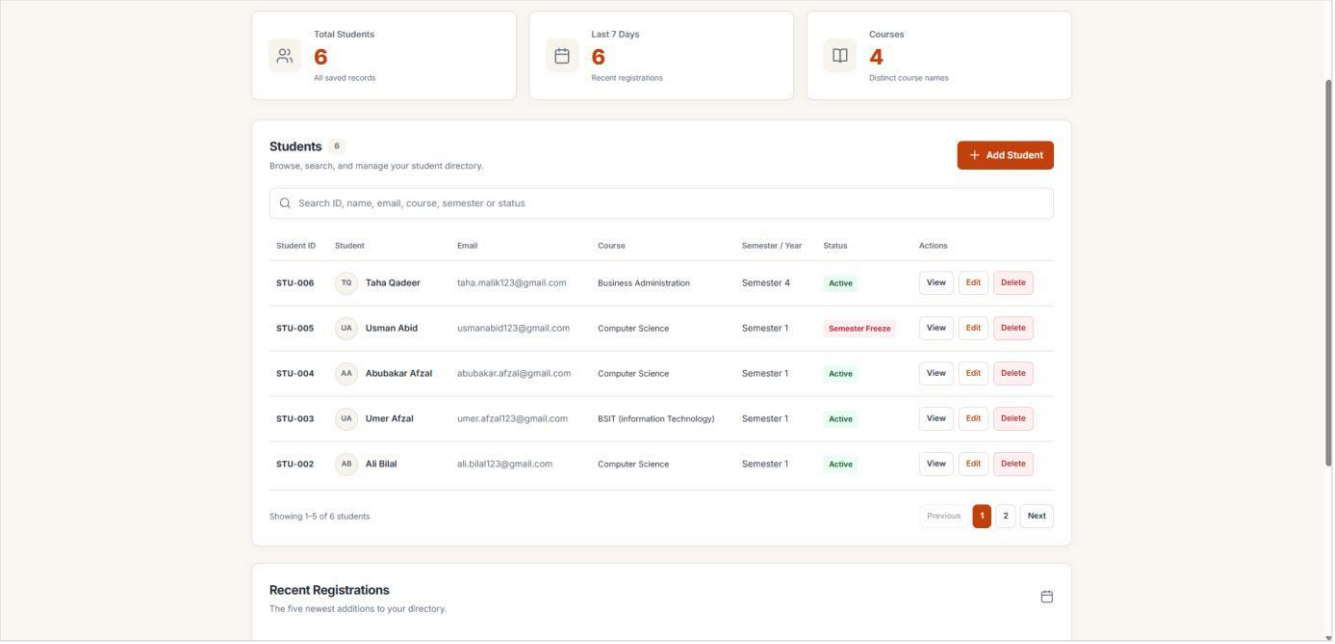
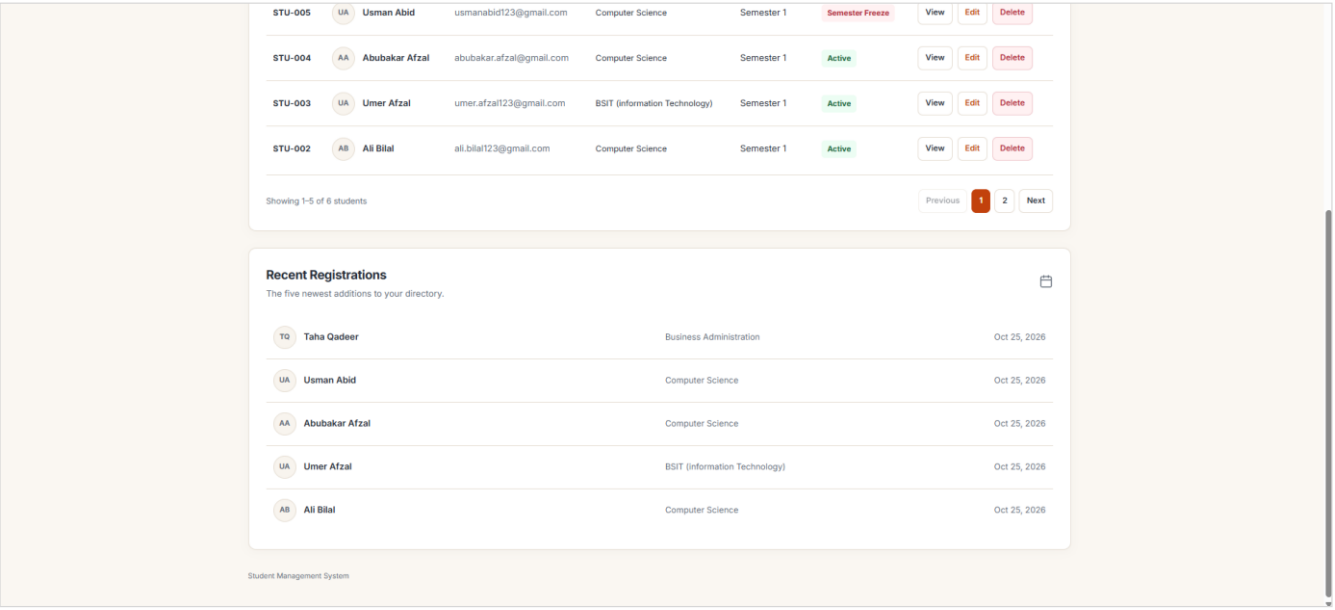


Figure 5b. Directory continuation and recent registrations



Search filters the directory, pagination displays five records per page, and recent registrations show the newest additions.

Student Operations

Figure 6. Add Student form

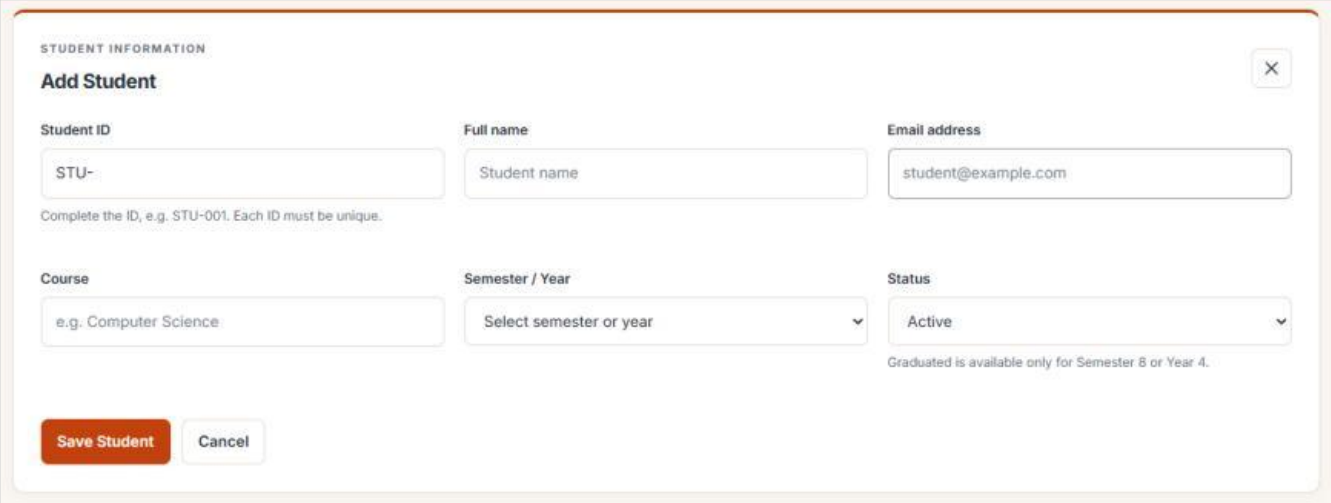


Figure 7. Edit Student form and status options

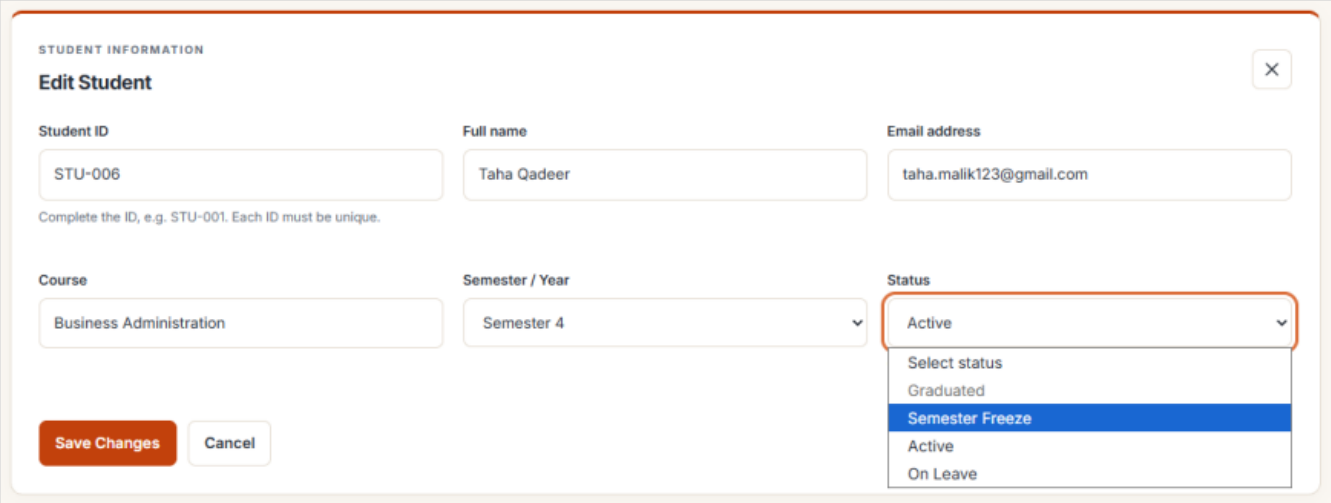
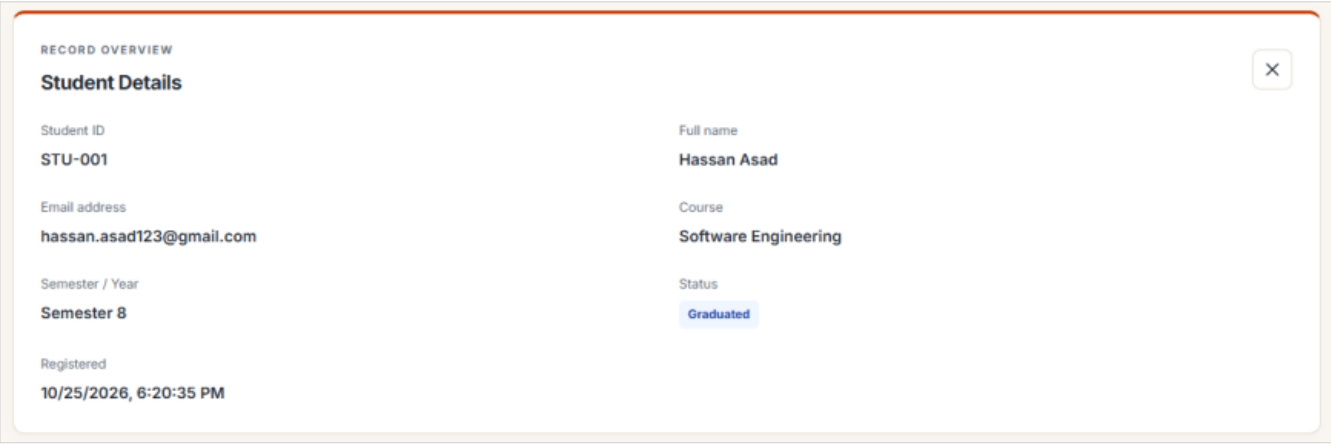


Figure 8. Student details



Mobile Layout and Themes

Figure 9. Mobile - light mode

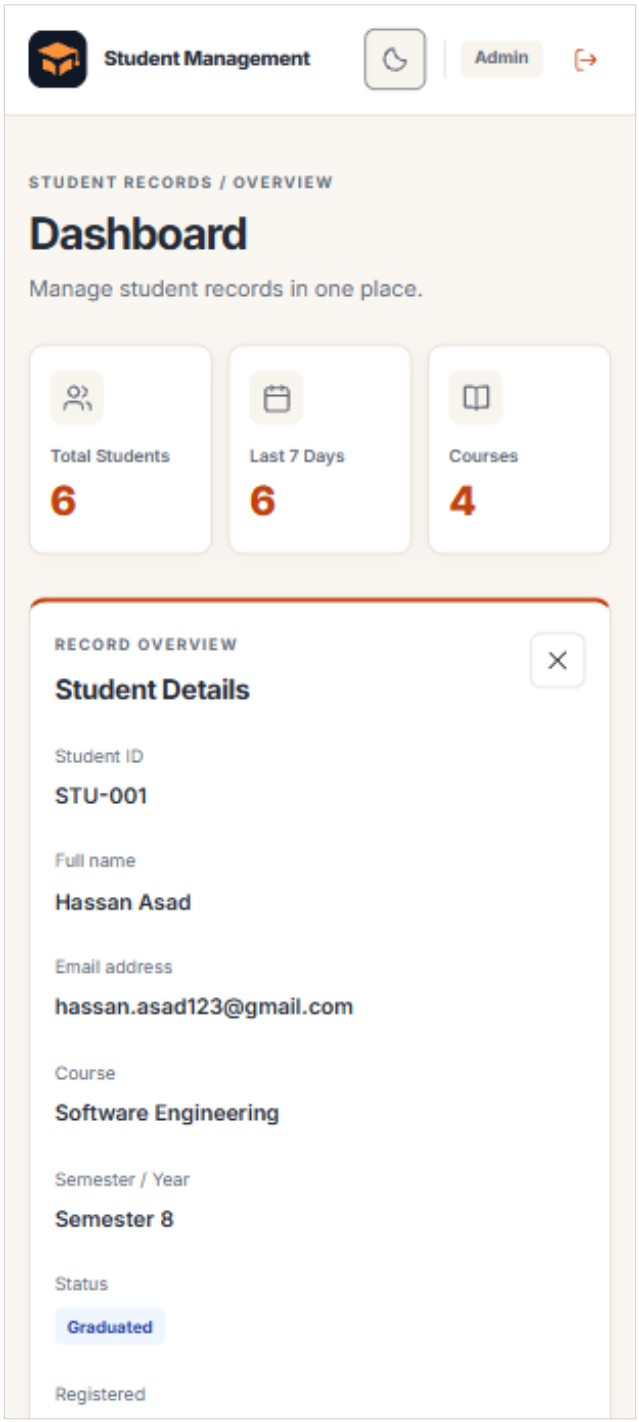
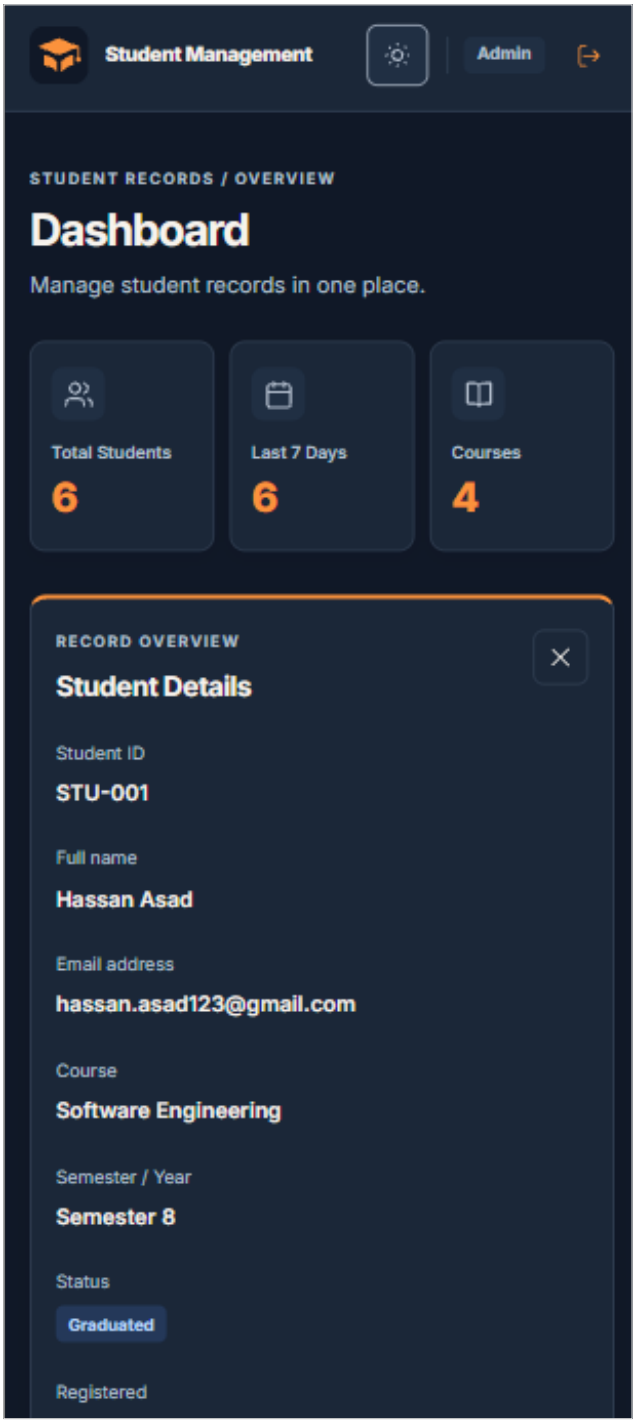


Figure 10. Mobile - dark mode



The narrow-screen layout stacks student information vertically. Light and dark themes share the same content and controls, and the selected theme is saved.

Code Screenshots

Figure 11. React API request and error handling

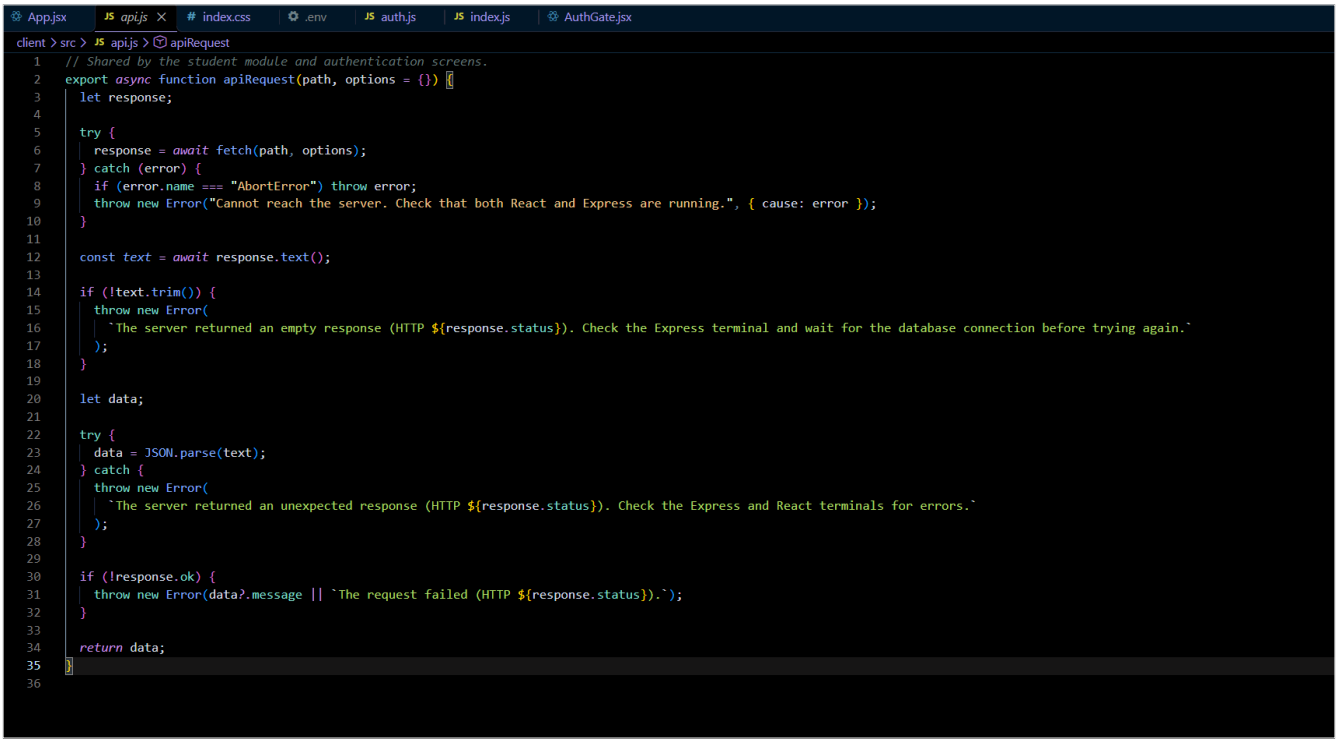
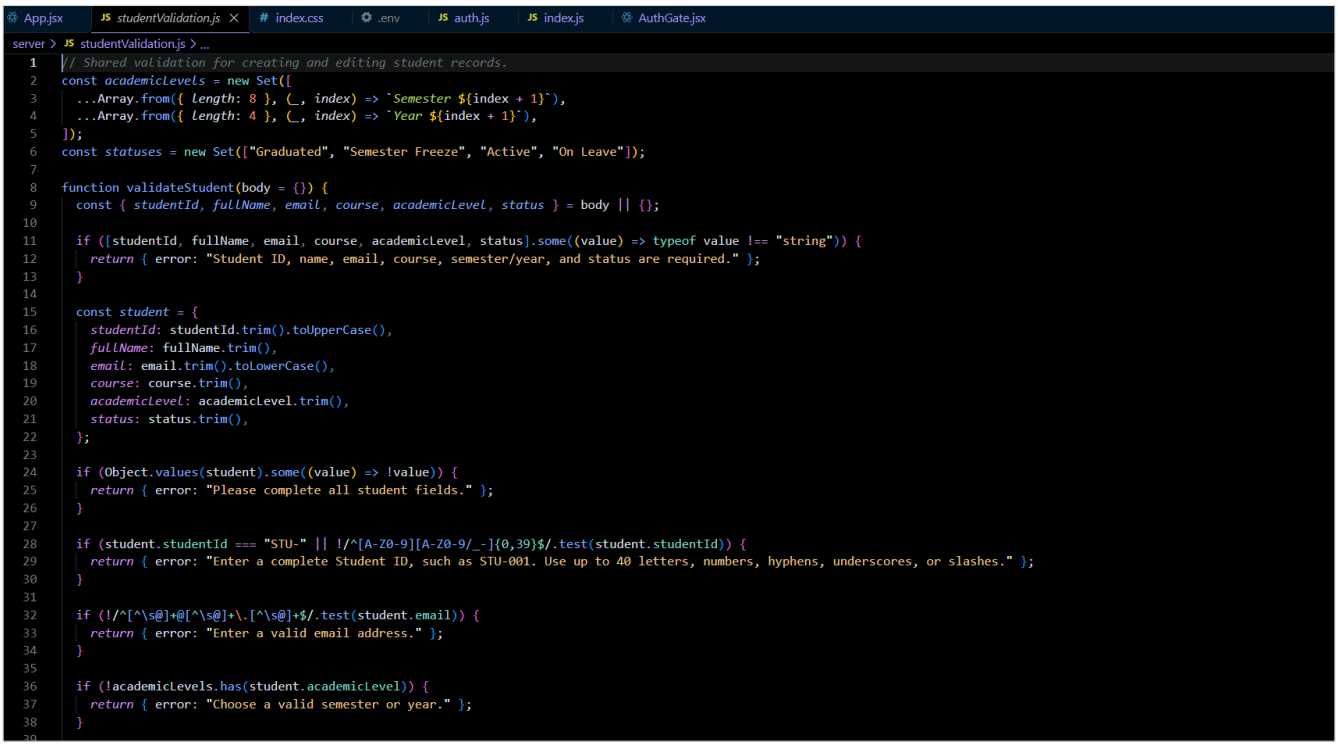
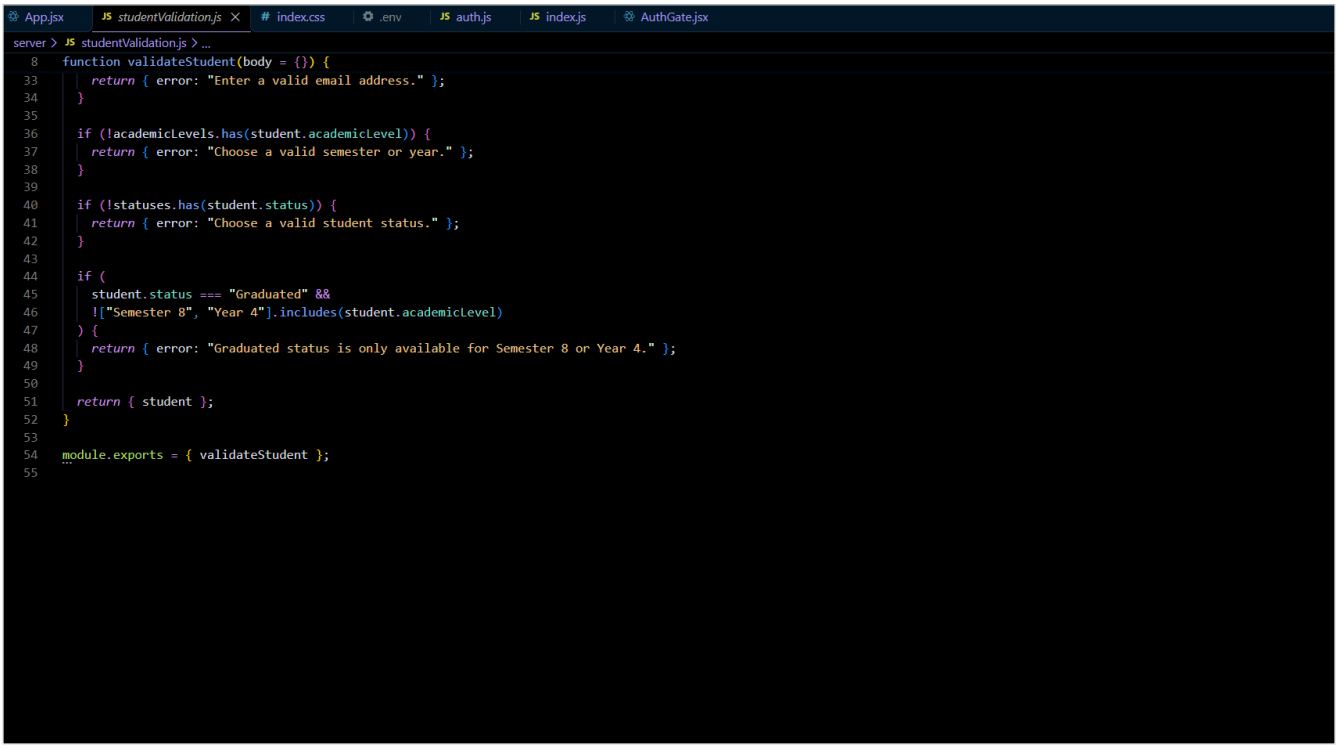


Figure 12a. Backend student field validation



Validation and Conclusion

Figure 12b. Semester, status, and graduation validation



Testing Summary

The frontend passed ESLint and a production build. Backend syntax checks and isolated validation checks covered Student IDs, duplicate-conflict handling, semester/year choices, status options, and graduation rules. Functional testing reported by the student covered persistent storage, student operations, authentication, roles, search, and pagination.

Conclusion

The project fulfills the specified Student Management System features and both bonuses using the MERN stack. It connects a responsive React interface to an Express API and MongoDB, with validation and role checks on the backend. Deployment remains outside the current submission scope.

GitHub Repository

https://github.com/HassanAsad118/Student-Management-System